Oil Price Prediction Using LSTM and Simple RNN

Yasir Abdullah Aladwani

Project Report - 2025

Contents

Section	Page
Executive Summary	3
Project Overview and Dataset	4
Data Preparation and Methodology	5
Exploratory Data Analysis I	6
Exploratory Data Analysis II	7
Exploratory Data Analysis III	8
How the Models Work and RNN Challenges	9
LSTM Results	10
Simple RNN Results	11
Final Comparison	12
Side-by-Side Forecast Comparison	13
Conclusion	14

Executive Summary

This report presents a comparative time-series forecasting study for crude oil prices using two recurrent deep learning models: LSTM and Simple RNN. Both models were trained on the same historical dataset and evaluated under the same preprocessing and splitting strategy to ensure a fair comparison. The goal was to predict the next closing price using the previous 60 days of closing-price observations.

Model	MAE	RMSE	R^2
LSTM	1.6842	2.3011	0.9636
Simple RNN	3.3420	4.3323	0.8710

Main Finding: The LSTM model clearly outperformed the Simple RNN model by achieving lower prediction errors and a higher coefficient of determination. This indicates that LSTM captured temporal dependencies in the oil-price series more effectively.

Project Significance: Accurate oil-price forecasting is valuable for energy-market analysis, budgeting, strategic planning, and risk-aware decision support. The comparison in this report also highlights why advanced recurrent models are often preferred over basic recurrent baselines when long-term dependencies matter.

Project Overview and Dataset

The project focuses on forecasting crude oil closing prices using historical market data. It compares two recurrent neural-network approaches on the same task and with the same evaluation pipeline in order to identify the better forecasting model.

The dataset used in this project is an open-source dataset from Kaggle.

Dataset Profile

- Number of rows: **6,073**
- Number of columns: **7**
- Columns observed in the notebook: Date, Adj_Close, Close, High, Low, Open, Volume
- Missing values: **No missing values were found**
- Forecast target used for modeling: **Close**

The dataset was first sorted chronologically by date and then inspected through summary statistics. The descriptive analysis confirmed a broad range of price values over the years, including periods of high volatility. The modeling task in the notebook was intentionally simplified to a univariate forecasting setting in which only the closing price was used as the target sequence.

Data Preparation and Methodology

The notebook follows a clear and reproducible workflow for time-series forecasting. Each step below was verified directly from the notebook to ensure that the report remains fully consistent with the original implementation.

Step	Verified Details from the Notebook
Chronological Ordering	The Date column was converted to datetime format and the data was sorted in ascending chronological order.
Target Definition	Only the Close price was used for univariate forecasting.
Look-back Window	Each input sample used the previous 60 days to predict the next value.
Train / Validation / Test Split	70% train, 15% validation, and 15% test.
Scaling	MinMaxScaler was fitted on the training set only to reduce data leakage.
Input Shapes	X_train = (4191, 60, 1), X_val = (910, 60, 1), and X_test = (912, 60, 1).
Training Setup	Both models used the Adam optimizer and mean squared error loss, with batch size 32, up to 50 epochs, and early stopping to reduce overfitting.

Exploratory Data Analysis

The first stage of exploratory analysis focuses on the long-term trend and a simple seasonal view of prices.

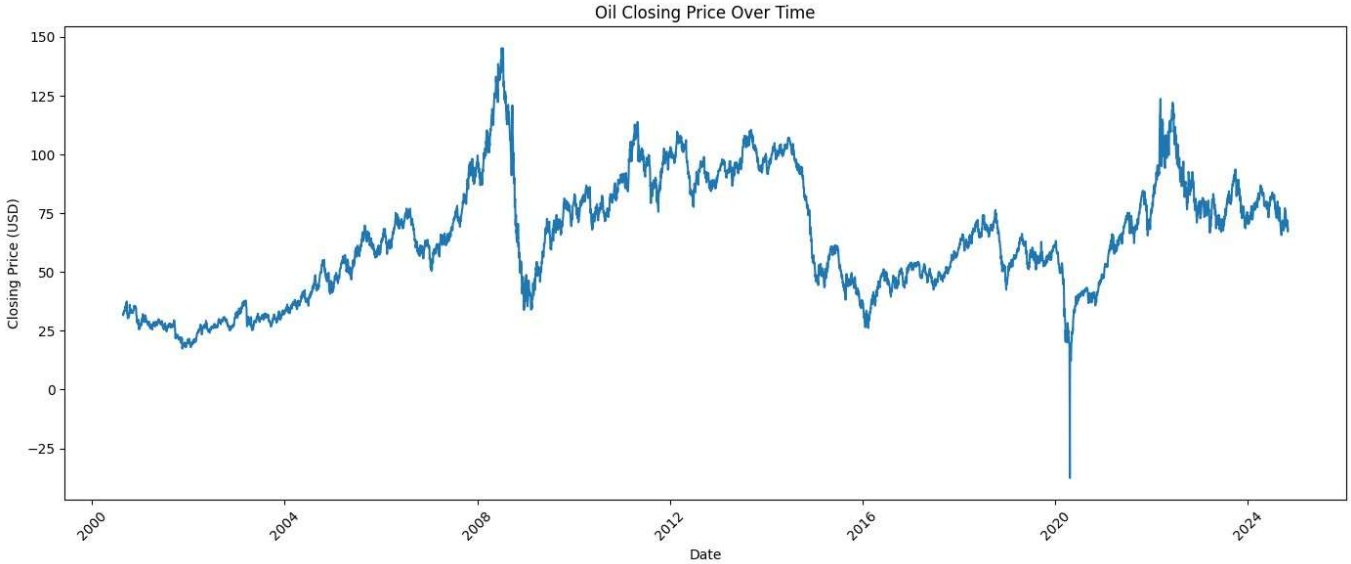


Figure 1. Oil closing price over time.

Figure 2. Average oil closing price by month.

The next visualizations highlight average yearly movement and short- versus long-term smoothing using moving averages.

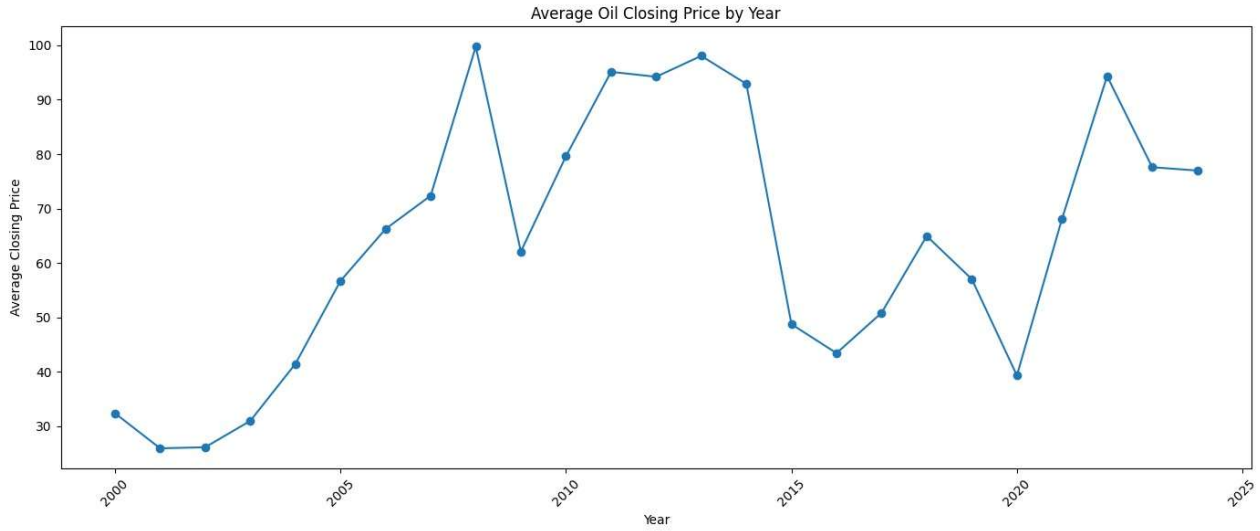


Figure 3. Average oil closing price by year.

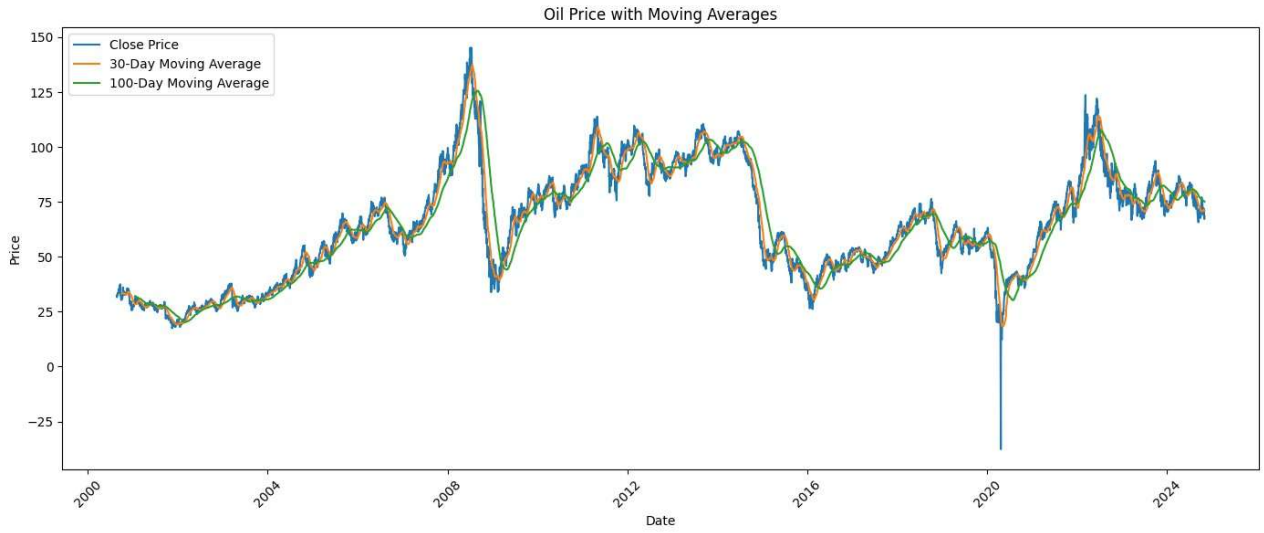


Figure 4. Closing price with 30-day and 100-day moving averages.

Volatility was further explored through the daily high-low difference, which provides a simple view of intraday price spread behavior over time.

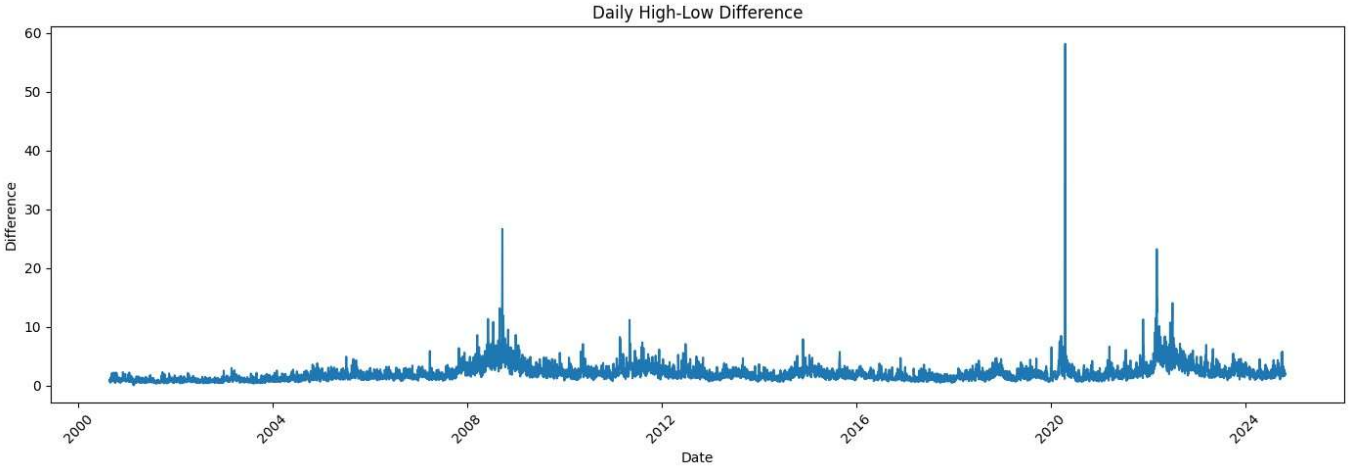


Figure 5. Daily high-low difference.

Together, the EDA results show that oil prices are highly dynamic and experience both long-term regime shifts and shorter-term fluctuations. This pattern makes the task well suited for recurrent sequence models.

How the Models Work and RNN Challenges

How a Simple RNN Works	How an LSTM Works
A Simple RNN processes a sequence one time step at a time. At each step, it combines the current input with the hidden state from the previous step to produce a new hidden state. That hidden state acts as the model's memory and is then passed forward through the sequence. This design makes Simple RNN a useful baseline for sequence modeling, but its memory is fragile when the dependency becomes long.	An LSTM extends the basic recurrent idea by introducing a cell state and three gates: forget, input, and output. These gates decide what information should be removed, updated, and exposed at each time step. Because the cell state can carry useful information across longer horizons, LSTM is better suited for learning long-term dependencies in time-series data.

A key limitation of standard RNNs is gradient instability during backpropagation through time:

- **Vanishing gradient:** the gradient becomes extremely small as it is propagated backward across many time steps, so early information is gradually forgotten and long-range dependencies become difficult to learn.

- **Exploding gradient:** the gradient becomes excessively large, leading to unstable weight updates and difficult optimization.

LSTM was specifically designed to reduce the vanishing-gradient problem by regulating information flow through its gates and cell state. The diagram below illustrates why LSTMs preserve useful influence across more time steps than standard RNNs.

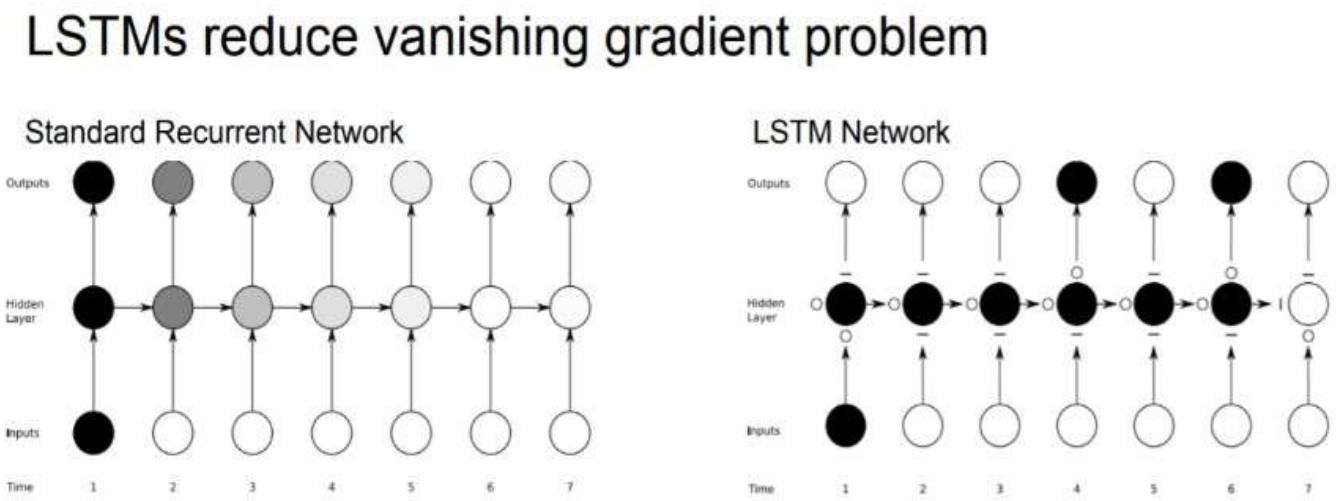


Figure 6. Illustration of how LSTMs reduce the vanishing-gradient problem compared with standard recurrent networks.

LSTM Results

The LSTM model achieved the strongest forecasting performance in the notebook. Its error values were substantially lower than those of the Simple RNN model, and its R^2 value indicates a strong fit between the predicted and actual oil prices on the test set.

Metric	Value
MAE	1.6842
RMSE	2.3011
R^2	0.9636

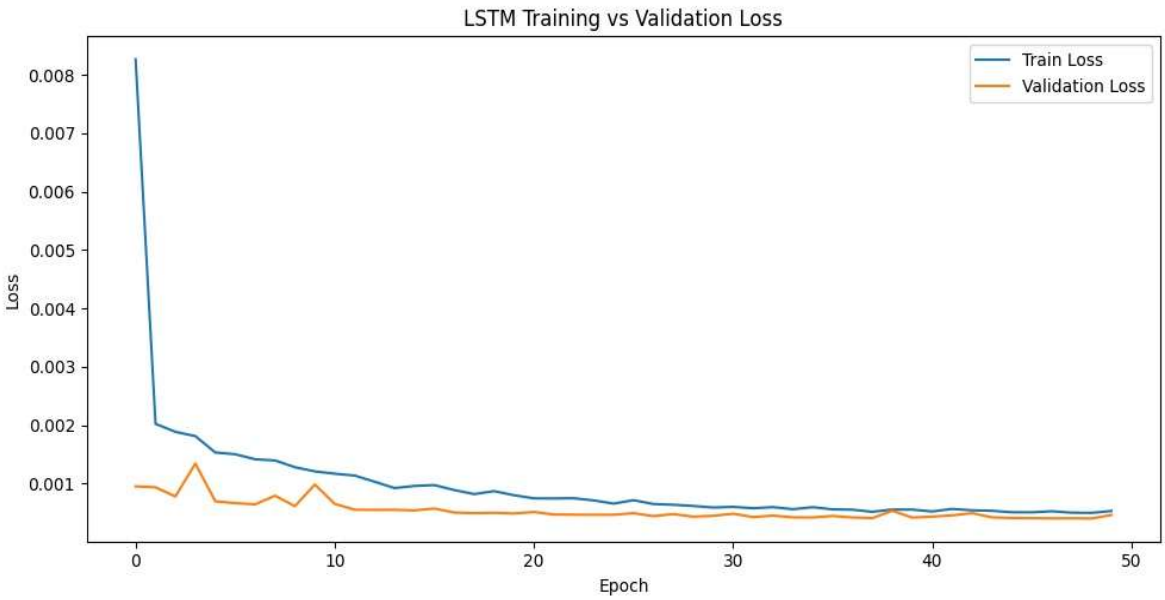


Figure 7. LSTM training vs validation loss.

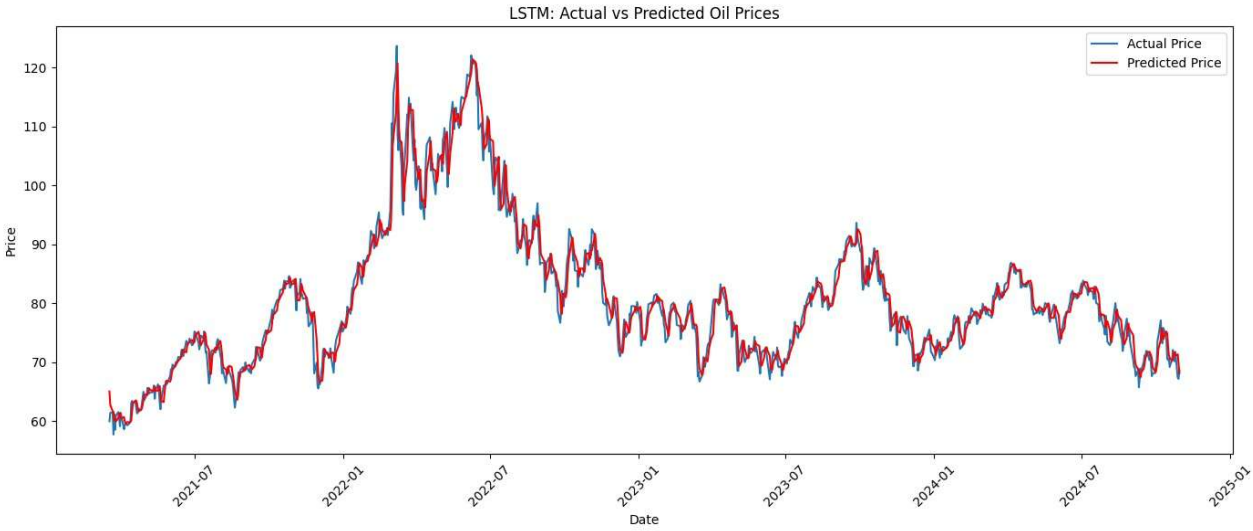


Figure 8. LSTM actual vs predicted oil prices.

Simple RNN Results

The Simple RNN model was able to track the general movement of the oil-price series, but it produced larger errors than LSTM. This is consistent with the known limitation of basic recurrent networks when the prediction task depends on longer temporal context.

Metric	Value
MAE	3.3420
RMSE	4.3323
R^2	0.8710

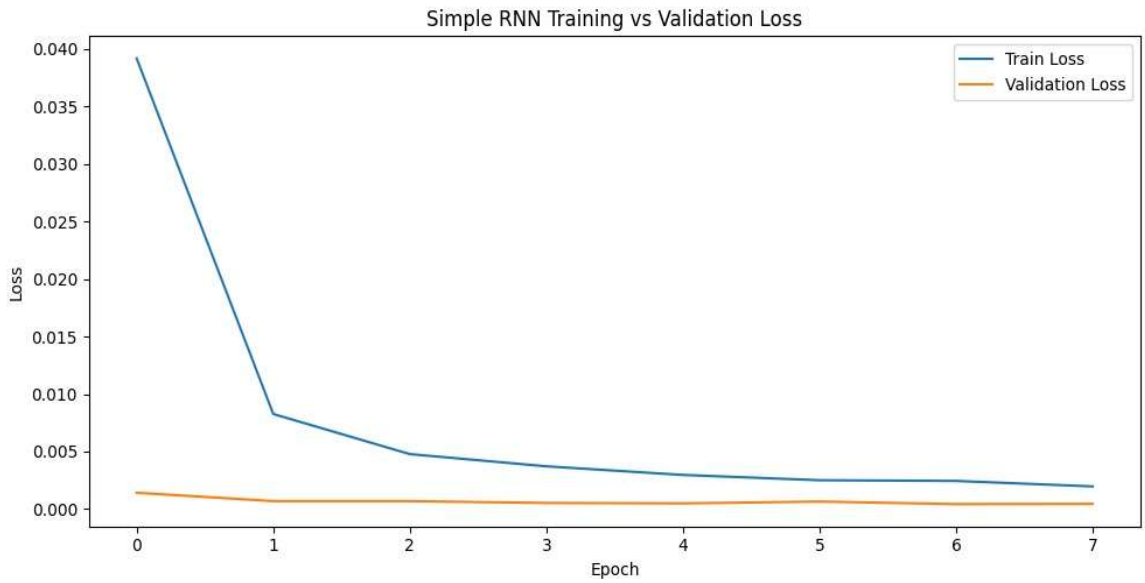


Figure 9. Simple RNN training vs validation loss.

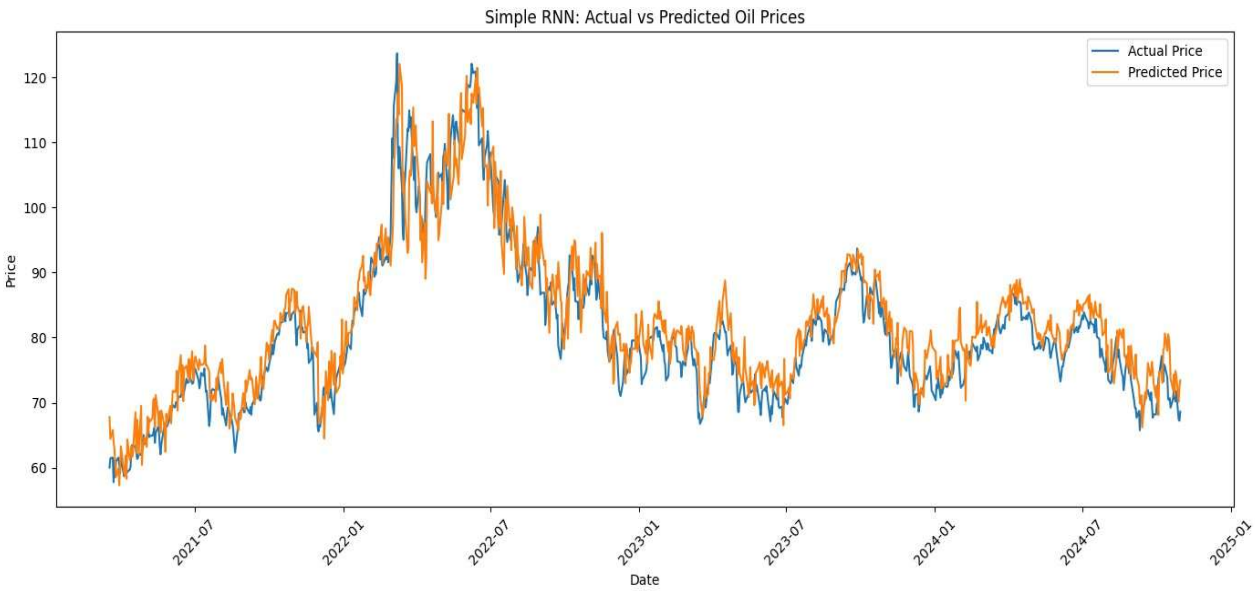


Figure 10. Simple RNN actual vs predicted oil prices.

Final Comparison

The table below summarizes the final evaluation results extracted directly from the notebook. Lower MAE and RMSE indicate better forecasting accuracy, while higher R^2 indicates better explanatory power.

Model	MAE	RMSE	R^2
LSTM	1.6842	2.3011	0.9636
Simple RNN	3.3420	4.3323	0.8710

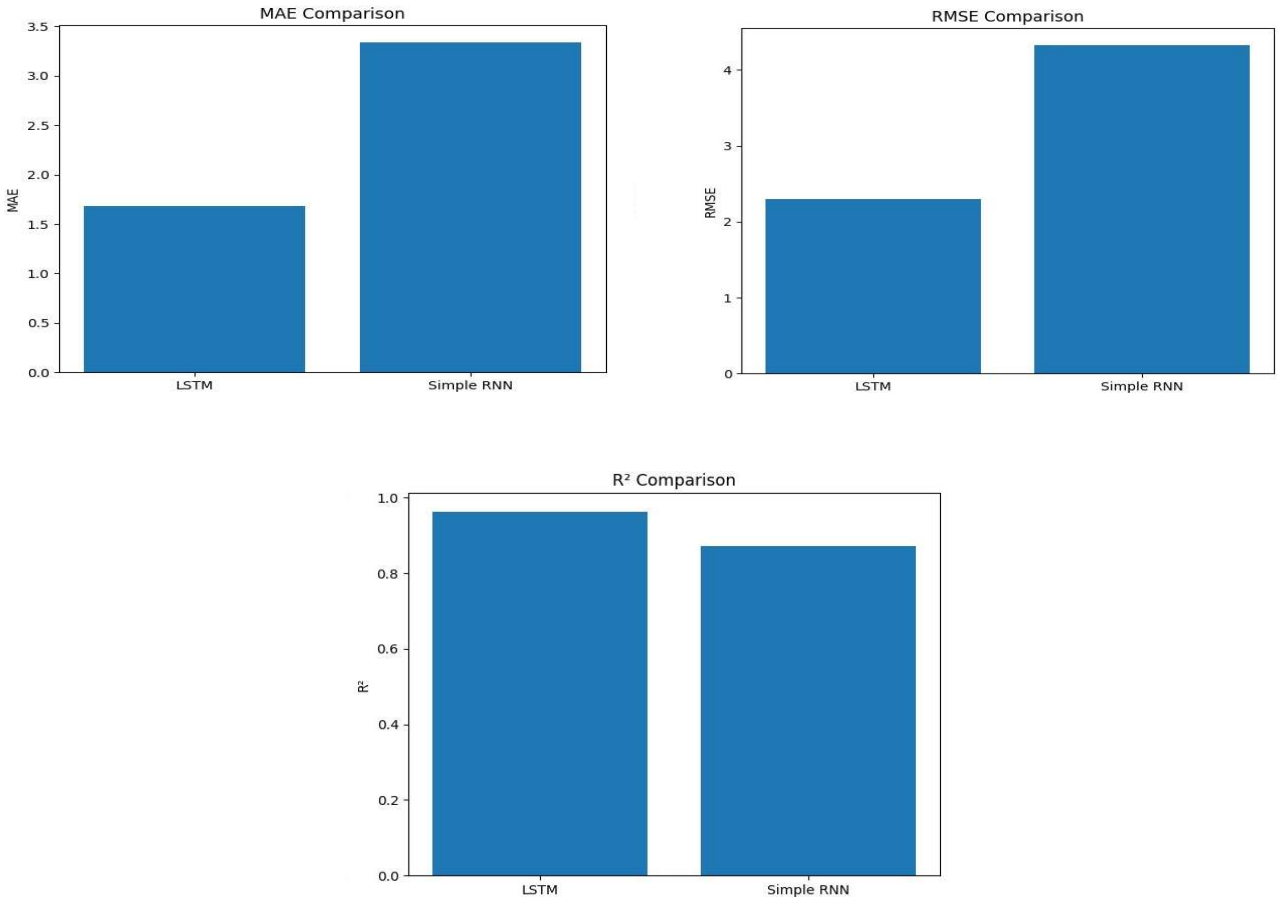
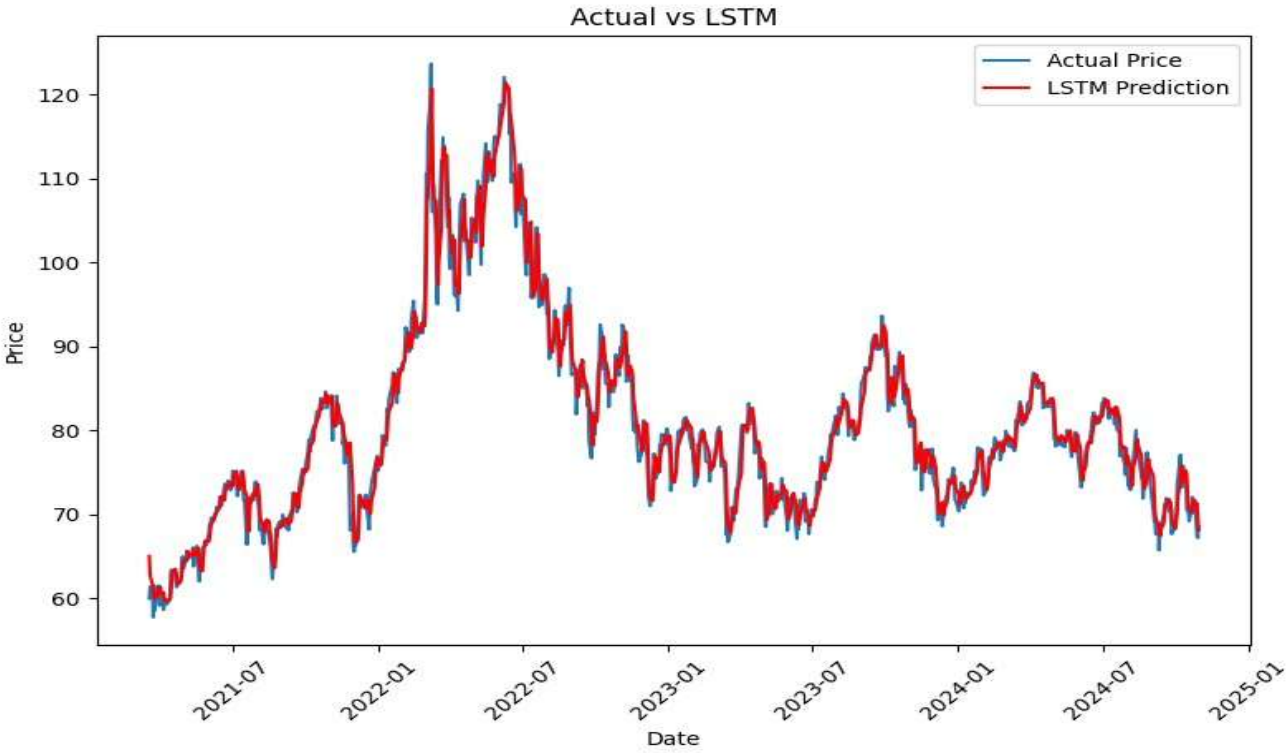


Figure 11. Comparison bar charts for MAE, RMSE, and R^2.

Best model based on RMSE: LSTM (RMSE = 2.3011).

Side-by-Side Forecast Comparison

The combined plot below provides a direct visual comparison between the actual test values and each model's predictions. It further confirms that the LSTM prediction line follows the actual series more closely than the Simple RNN prediction line.



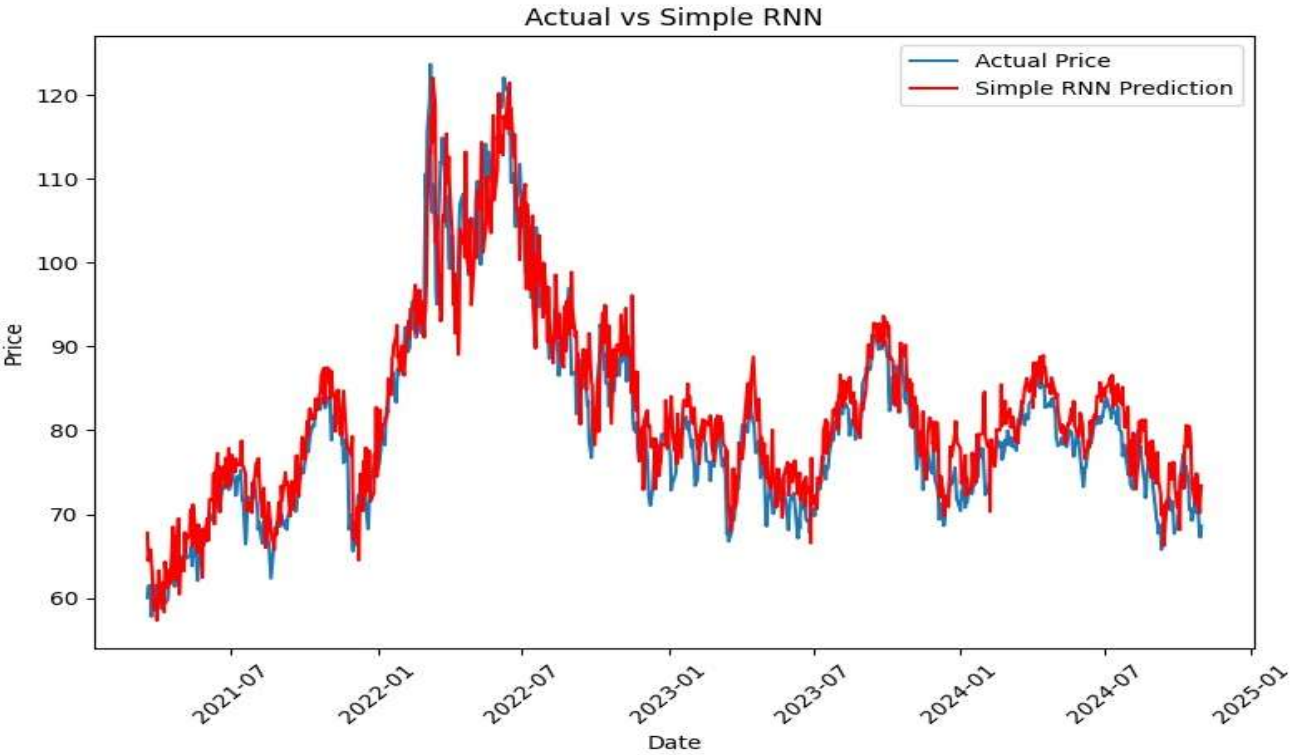


Figure 12. Actual price against LSTM and Simple RNN predictions.

Conclusion

This project demonstrated a clean and effective comparison between LSTM and Simple RNN for oil-price forecasting. After verifying all metrics and figures from the notebook, the evidence clearly shows that LSTM is the stronger model for this task.

Key Conclusions
1. Both models were trained under the same preprocessing and evaluation pipeline, which makes the comparison fair.
2. LSTM achieved the best overall performance with the lowest MAE and RMSE and the highest R^2.
3. The superior LSTM performance is consistent with its ability to retain useful long-term information through gated memory mechanisms.
4. Simple RNN remains a useful baseline, but it is less robust for long temporal dependencies because of gradient instability issues such as vanishing and exploding gradients.
5. For future improvement, the project could be extended with multivariate inputs such as macroeconomic indicators, news sentiment, or supply-demand variables.

Final Recommendation: LSTM should be selected as the final forecasting model because it consistently delivered more accurate and more stable predictions than the Simple RNN baseline.

GitHub Repository:

https://github.com/Yasir-Aladwani/Oil-Price-Prediction-LSTM-RNN.git